



Sir Syed University of Engineering & Technology, Karachi

Software Engineering (CS320)

Chapter-2

Software Process and Process Models

Topics Covered

- Software Process Framework
- Generic Process Framework Activities
- Software Process Models
- Build n Fix Model
- Water Fall Model

Overview

- **What? A software process** – as a framework for the tasks that are required to build high-quality software.
- **Who?** Managers, software engineers, and customers.
- **Why?** Provides stability, control, and organization to an activity
- **Steps?** A handful of activities are common to all software processes, details vary.
- **Work product?** Programs, documents, and data

Process:

- It's a foundation layer for software engineering.
- It's define **framework** for a set of *key process areas* (KRA) for effectively manage and deliver quality software in a cost effective manner
- The processes define the tasks to be performed and the order in which they are to be performed

Process (Continue)

- A software development methodology (also known as a system development methodology, software development life cycle, software development process, software process) is a division of software development work into distinct phases (or stages) containing activities with the intent of better planning and management.

Process (Continue)

- Purpose of process is to deliver software in a timely manner and with sufficient quality to satisfy those who have sponsored its creation and those who will use it.
- The methodology may include the pre-definition of specific deliverables and artifacts that are created and completed by a project team to develop or maintain an application.

Process framework

Why process

A process defines who is doing what, when and how to reach a certain goal.

- To build complete software process.
- Identified a small number of framework activities that are applicable to all software projects, regardless of their size or complexity.
- It encompasses a set of umbrella activities that are applicable across the entire software process.

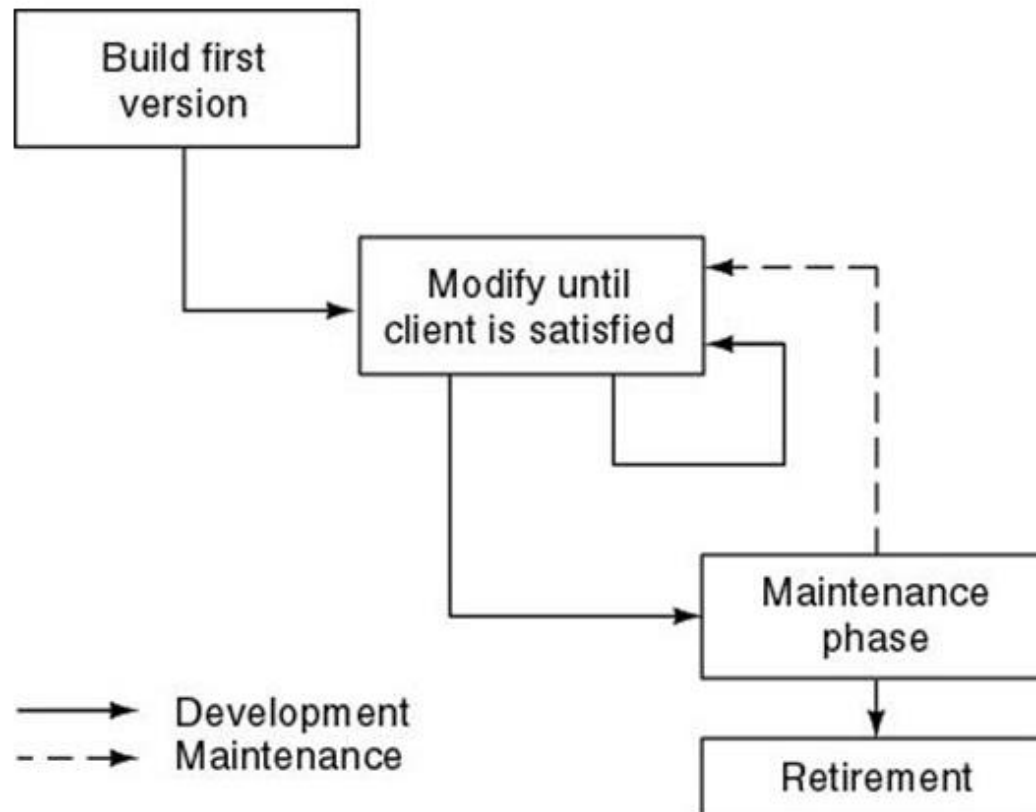
Generic Process Framework Activities

- **Communication:**
 - Heavy communication with customers, stakeholders, team
 - Encompasses requirements gathering and related activities
- **Planning:**
 - Workflow that is to follow
 - Describe technical task, likely risk, resources will require, work products to be produced and a work schedule.
- **Modeling:**
 - Help developer and customer to understand requirements (Analysis of requirements) & Design of software
- **Construction**
 - Code generation: either manual or automated or both
 - Testing – to uncover error in the code.
- **Deployment:**
 - Delivery to the customer for evaluation
 - Customer provide feedback

Software process model

- Process models prescribe a distinct set of activities, actions, tasks, milestones, and work products required to engineer high quality software.
- Process models are not perfect, but provide roadmap for software engineering work.
- Software models provide stability, control, and organization to a process that if not managed can easily get out of control
- Software process models are adapted to meet the needs of software engineers and managers for a specific project.

Build and Fix Model



Build and Fix Model

The earlier approach

- Product is constructed without specification or any attempt at design.
- developers simply build a product that is reworked as many times as necessary to satisfy the client.
- model may work for small projects but is totally unsatisfactory for products of any reasonable size.
- Maintenance is high.
- Source of difficulties and deficiencies
 - impossible to predict
 - impossible to manage

Why Models are needed?

- Symptoms of inadequacy: the software crisis
 - scheduled time and cost exceeded
 - user expectations not met
 - poor quality

Problems

- The assumption is that requirements can be fully understood prior to development
- Interaction with the customer occurs only at the beginning (requirements) and end (after delivery)
- Unfortunately the assumption almost never holds

Advantages

- Reduce risks by improving visibility
- Allow project changes as the project progresses
 - based on feedback from the customer

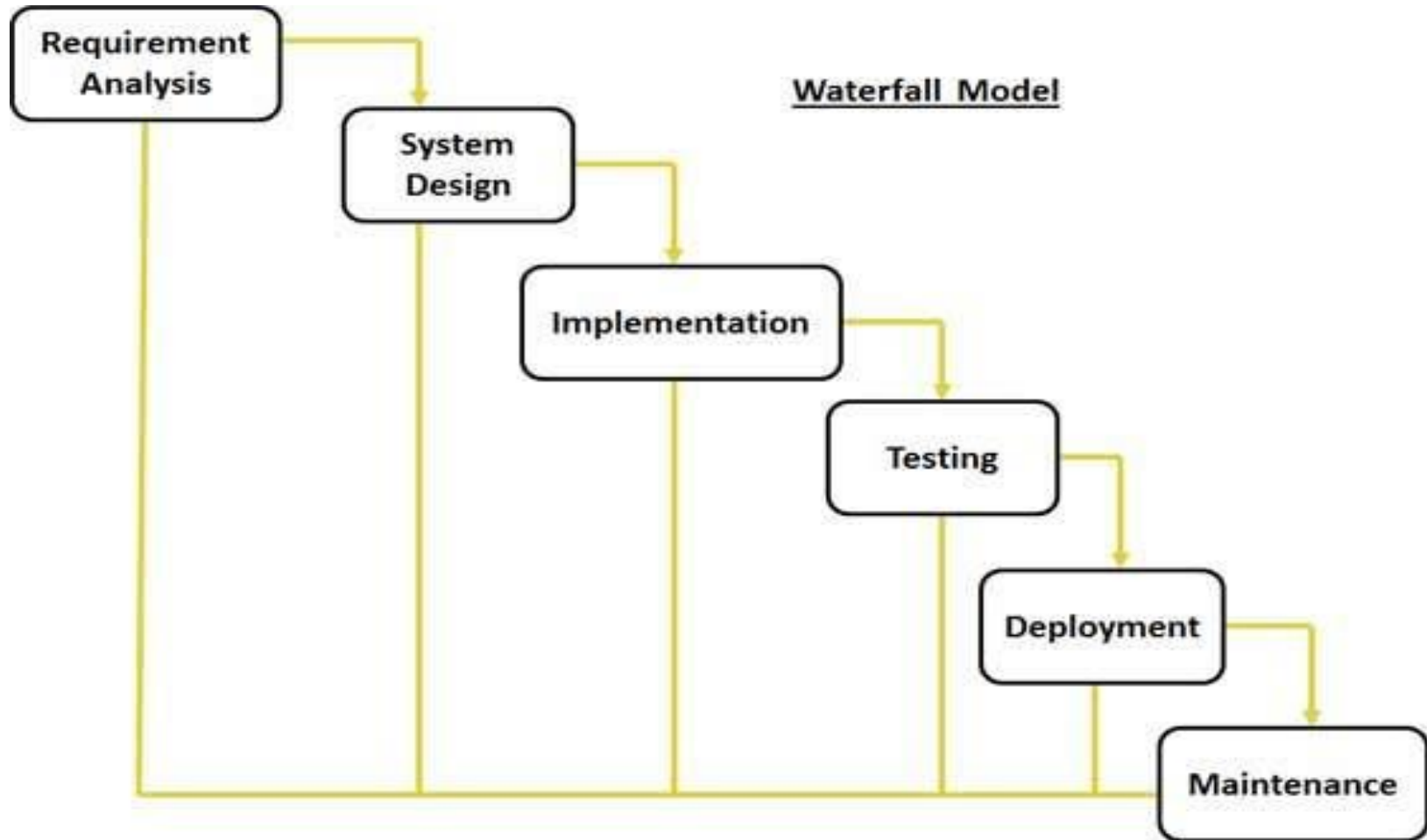
Prescriptive Model

- Prescriptive process models advocate an orderly approach to software engineering
 - Organize framework activities in a certain order
- Each action in terms of a task set that identifies the work to be accomplished to meet the goals.
- Software engineer choose process framework that includes activities like;
 - Communication
 - Planning
 - Modeling
 - Construction
 - Deployment

Prescriptive Model

- Calling this model as “Prescribe” because it recommend a set of process elements, activities, action task, work product & quality.
- Each elements are inter related to one another (called workflow).

Waterfall Model



Waterfall Model

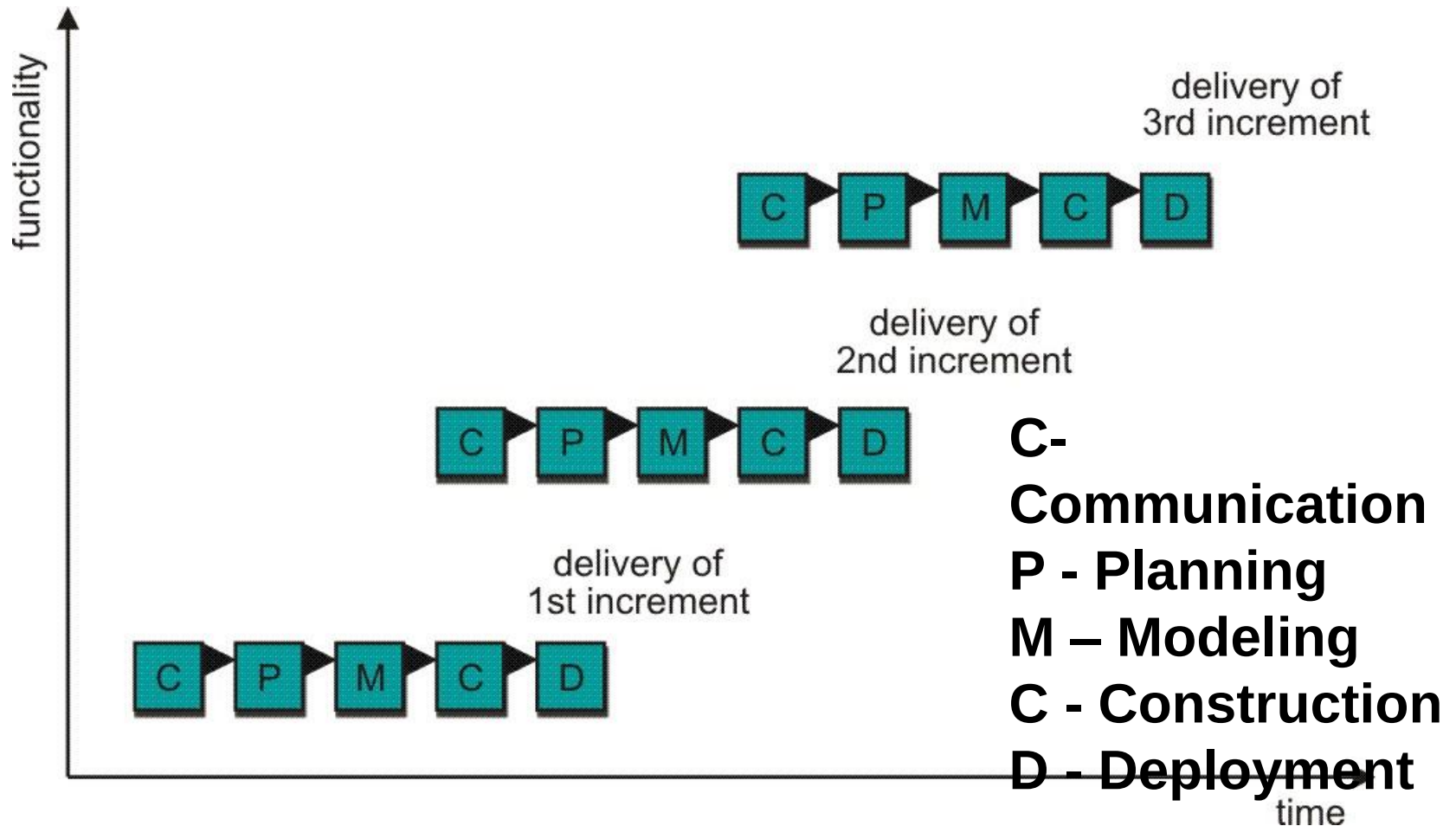
The sequential phases in Waterfall model are:

- **Requirement Gathering and analysis:** All possible requirements of the system to be developed are captured in this phase and documented in a requirement specification doc.
- **System Design:** The requirement specifications from first phase are studied in this phase and system design is prepared. System Design helps in specifying hardware and system requirements and also helps in defining overall system architecture.
- **Implementation:** With inputs from system design, the system is first developed in small programs called units, which are integrated in the next phase. Each unit is developed and tested for its functionality which is referred to as Unit Testing.

Waterfall Model

- **Integration and Testing:** All the units developed in the implementation phase are integrated into a system after testing of each unit. Post integration the entire system is tested for any faults and failures.
- **Deployment of system:** Once the functional and non functional testing is done, the product is deployed in the customer environment or released into the market.
- **Maintenance:** There are some issues which come up in the client environment. To fix those issues patches are released. Also to enhance the product some better versions are released. Maintenance is done to deliver these changes in the customer environment.

Incremental Process Model



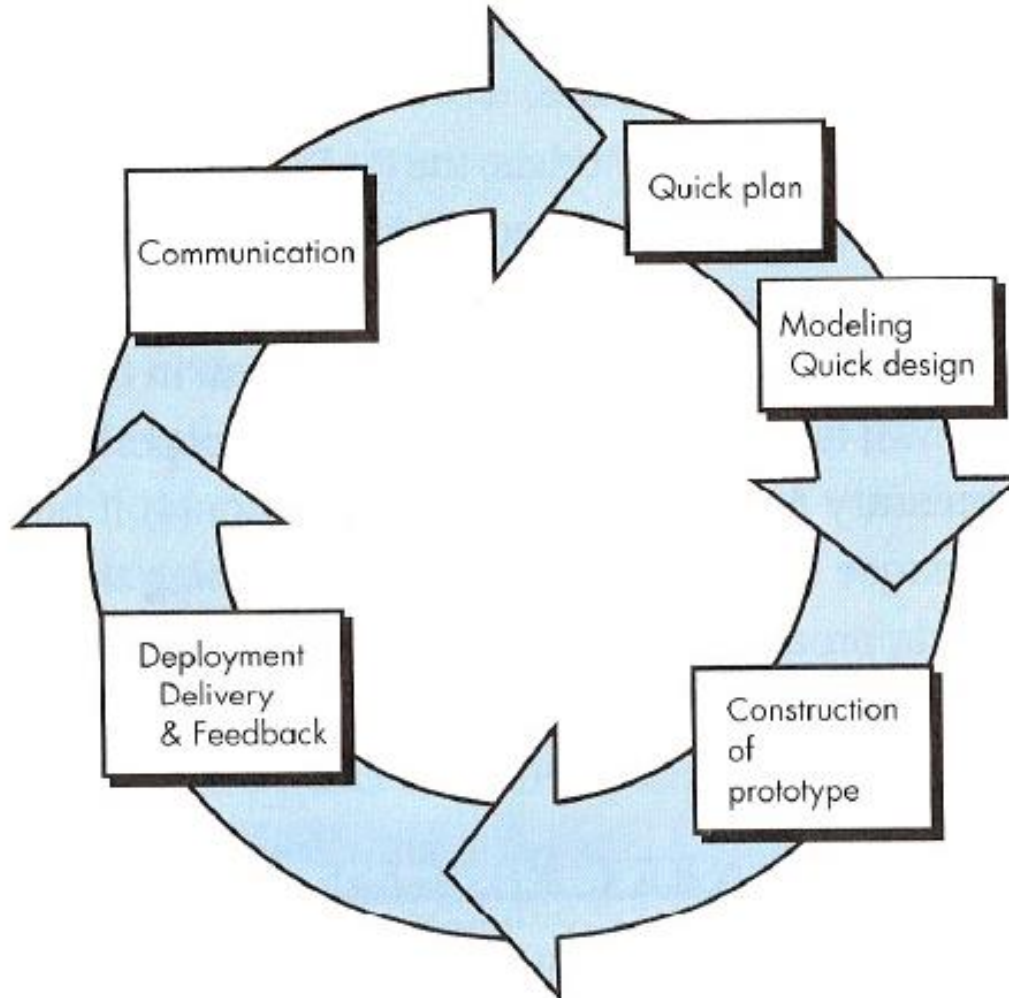
Incremental Process Model

- Rather than deliver the system as a single delivery, the development and delivery is broken down into increments with each increment delivering part of the required functionality.
- First Increment is often core product
 - Includes basic requirement
 - Many supplementary features (known & unknown) remain undelivered
- A plan of next increment is prepared
 - Modifications of the first increment
 - Additional features of the first increment
- It is particularly useful when enough staffing is not available for the whole project
- Incremental model focus more on delivery of operation product with each increment.

Evolutionary Process Models

- Produce an increasingly more complete version of the software with each iteration.
- Evolutionary Models are iterative.
- Evolutionary models are:
 - Prototyping
 - Spiral Model
 - Concurrent Development Model

Evolutionary Process Models



Prototype Model

- **Best approach when:**
 - Objectives defined by customer are general but does not have details like input, processing, or output requirement.
 - Developer may be unsure of the efficiency of an algorithm, O.S., or the form that human machine interaction should take.
- It can be used as standalone process model.
- Model assist software engineer and customer to better understand what is to be built when requirement are fuzzy.
- Prototyping start with communication, between a customer and software engineer to define overall objective, identify requirements and make a boundary.
- Going ahead, planned quickly and modeling (software layout visible to the customers/end-user) occurs

Prototype Model

- Quick design leads to prototype construction.
 - Prototype is deployed and evaluated by the customer/user.
 - Feedback from customer/end user will refine requirement and that is how iteration occurs during prototype to satisfy the needs of the customer.
-
- Prototype can be serve as “the first system”.
 - Both customers and developers like the prototyping paradigm.
 - Customer/End user gets a feel for the actual system
 - Developer get to build something immediately.

Types of prototyping- Throwaway prototyping

Also called close-ended prototyping. **Throwaway** or Rapid Prototyping refers to the creation of a model that will eventually be discarded rather than becoming part of the final delivered software

- The most obvious reason for using Throwaway Prototyping is that it can be done quickly
- Another strength of Throwaway Prototyping is its ability to construct interfaces that the users can test.

Types of prototyping- Throwaway prototyping

In this approach the prototype is constructed with the idea that it will be discarded and the final system will be built from scratch. The steps in this approach are:

- Write preliminary requirements
- Design the prototype
- User experiences/uses the prototype, specifies new requirements
- Repeat if necessary
- Write the final requirements

Types of prototyping- Evolutionary Prototyping

- Evolutionary Prototyping is quite different from Throwaway Prototyping.
- The main goal when using Evolutionary Prototyping is to build a very robust prototype in a structured manner and constantly refine it.
- The reason for this is that the Evolutionary prototype, when built, forms the heart of the new system, and the improvements and further requirements will be built.
- When developing a system using Evolutionary Prototyping, the system is continually refined and rebuilt.

"...evolutionary prototyping acknowledges that we do not understand all the requirements and builds only those that are well understood"

Types of prototyping-Incremental Prototyping

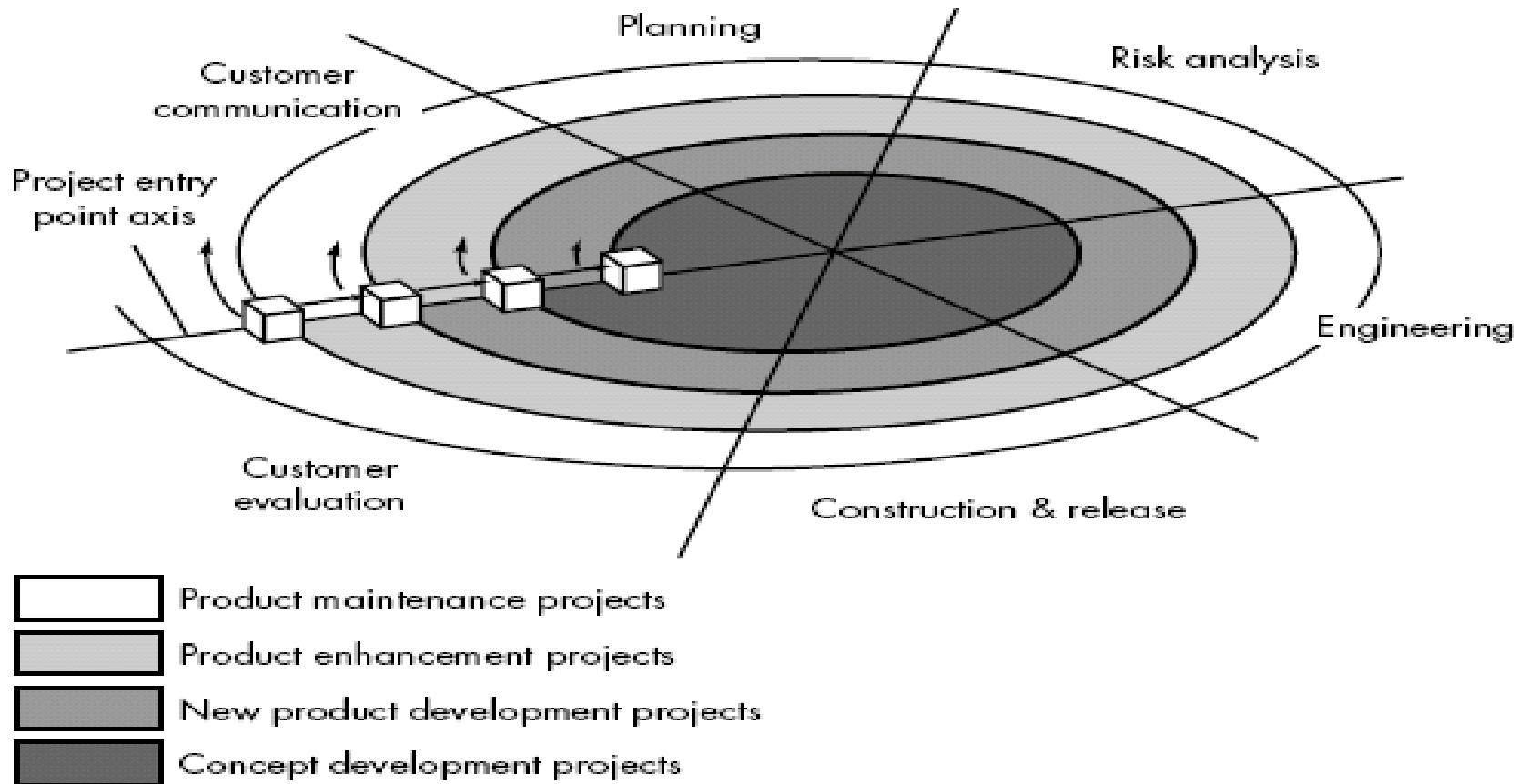
- The final product is built as separate prototypes. At the end the separate prototypes are merged in an overall design.
- By the help of incremental prototyping we can reduce the time gap between user and software developer

Spiral Model

Couples iterative nature of prototyping with the controlled and systematic aspects of the linear sequential model

- It provide potential for rapid development of increasingly more complete version of the software.
- Using spiral, software developed in as series of evolutionary release.
 - Early iteration, release might be on paper or prototype.
 - Later iteration, more complete version of software.
- Divided into framework activities (C,P,M,C,D). Each activity represent one segment.
- Evolutionary process begins in a clockwise direction, beginning at the center risk.

Spiral Model



Spiral Model

- **Planning Phase:** Requirements are gathered during the planning phase. Requirements like 'BRS' that is 'Business Requirement Specifications' and 'SRS' that is 'System Requirement specifications'.
- **Risk Analysis:** In the **risk analysis phase**, a process is undertaken to identify risk and alternate solutions. A prototype is produced at the end of the risk analysis phase. If any risk is found during the risk analysis then alternate solutions are suggested and implemented.

Spiral Model

- **Engineering Phase:** In this phase software is **developed**, along with testing at the end of the phase. Hence in this phase the development and testing is done.
- **Evaluation phase:** This phase allows the customer to evaluate the output of the project to date before the project continues to the next spiral.

Spiral Model

- Spiral models uses prototyping as a risk reduction mechanism but, more important, enables the developer to apply the prototyping approach at each stage in the evolution of the product.
- It maintains the systematic stepwise approach suggested by the classic life cycle but also incorporates it into an iterative framework activity.
- If risks cannot be resolved, project is immediately terminated